Software Design Project

Design Document

Mila Huovinen

Trang Vu

Van Bui

Ville Ylönen

Table of Contents

Software Design Project 1

Table of Contents 2

Introduction 3

Changelog 3

Quick Links 3

Overall Use-case 3

APIs Used in This Project 4

Technology Stack 5

- Programming languages 6
  - Primary programming language 6
  - Secondary programming languages 6
- Frameworks and libraries used 6
  - UI 6
  - Networking 6
  - Local Database 6
  - Build System 6
- The Usage of AI Tools in this Project 7
- Other Tooling Used in the Project 7
- Environment Requirements 7

Architecture and Components 7

- Overview 8

UI Prototype 8

Member Involvement 12

- Ville Ylönen 12
- Mila Huovinen 12
- Van Bui 13
- Trang Vu 13

Appendix 14

- Appendix A: API Test Scripts and JSON responses 14

Introduction

The primary purpose of this document describes the use-cases, technological decisions, overall architecture and individual components of this software project while also doubling as the project's changelog and a logbook of the group member's work. All in all, it will be a good starting point for someone unfamiliar with the project who wants to get to know the project. It will also act as a portal to all the other resources the project entails.

Change log

Version	Date	Notes
0.1	14.10.2026	Initial draft of the design document.
0.2.1	16.10.2026	Add details to all section and move the document from personal drive to the team's shared folder.
0.3	17.10.2026	Update Technology Stack section so that it contains the selected libraries and frameworks. Add section for listing the used APIs.
0.4	20.10.2026	Add more details to Overall Use-case section. Complete APIs Used in this Project section with actual API choice and actual test scripts and API responses.
0.5	23.10.2026	Add user stories to the Overall Use-case section. Add the UI prototype from Figma. Add the architecture and component design. Write the Usage of AI Tools and Other Tooling Used in the Project sections. Fix some minor wording issues.

Quick Links

This section contains links to other parts of the project.

Project repository: https://github.com/spakett/comp-se-110-project

UI Prototype: https://www.figma.com/design/JLui3o7nn19J3cZYxL5xUf/software-design?node-id=0-1&t=tYV8WZpaQpKTybxP-1

Overall Use-case

This section will explain the general idea and purpose of the program through both direct explanation and simple user stories.

Our idea is to implement a health-optimised commute planner that could help people with allergies or any other air-related health problems be able to plan their commuting plan ahead which is choosing the suitable route for their health condition.

Our application is planned to have two primary features:

1. A real-time heat-map of the concentrations of different allergens, pollutants and other irritants such as $PM_{2.5}$, PM_{10}, SO_2, NO_2, CO. This includes viewing historical data about the occurrence of these irritants.
2. A map tool for planning out routes that avoids a set of the aforementioned irritants.

The user would additionally be able to select and filter the specific irritants they want to affect the main functionalities of the application.

To clarify how the application is expected to be used, the following user stories describe the needs of our target users.

1. As a parent planning to take my children outside today, I want to see the overall air quality index across the city, so that I can choose the area and time with the cleanest air.
2. As a person with a birch pollen allergy, I want to see pollen levels on a map, so that I can avoid high-pollen areas while walking.
3. As a person with asthma, I want to select only the irritants that affect me, such as PM2.5 and NO_2, so that the heatmap and route suggestions focus on what matters for my health.
4. As a person sensitive to air pollution, I want to see how PM2.5 levels have changed over the past 30 days, so that I can understand whether today is better or worse than usual.
5. As a commuter who switches between walking, cycling and driving, I want to choose my travel mode, so that the suggested routes match how I will actually travel.

APIs Used in This Project

This section lists and explains the purpose of all the APIs used in this project.

In this project, we use four APIs to get data served for use case: Open-Meteo Air Quality API, Open-Meteo Weather Forecast API and Valhalla API and Nominatim API. From these APIs, the application will have air-quality-related data, environmental data, geocoding data and suggested routes.

Firstly, regarding air-quality data, Open-Meteo Air Quality API could help the application to request and obtain historical and forecasting hourly air-quality data with chosen location and time interval. These variables have been tested via requests: PM2.5, PM10, carbon monoxide, carbon dioxide, nitrogen dioxide, sulphur dioxide, ozone, methane, ammonia, UV index and some pollen. During implementation, if more or less pollutants, irritants or pollen are needed, then the number of variables should be changed, as long as they are supported by Open-Meteo Air Quality API. The retrieved data then directly appear in heatmap and affect the chosen commuting route for user.

Secondly, Open-Meteo Weather Forecast API helps the application to get weather condition data. This API also provides historical and forecast hourly and daily data of temperature, apparent temperature, relative humidity and maximum UV index. These environmental data is currently within scope of this project, however, if there are any changes, fetched data from Open-Meteo Weather Forecast API should also modify to be able to be visualized on heatmap or help internal route choosing logic.

Thirdly, Valhalla API is used to return the fastest route and alternatives of two chosen locations. Currently, this API is able to support three transportation modes: driving, cycling and walking, or three costing models as its API documentation stating, which makes it a great fit for this commuting planner application. Alternatives help the team to execute our internal route-optimised logic so that the suggested route for users should satisfy both fastest and safest criteria. However, in request body, it only accepts latitude and longitude of origin and destination, so it raises a need for another geocoding API which returns coordinates of existing location.

Nominatim Search API can fulfill this need. It could return the coordinates of user's addresses; with this API, user could experience actual map-like UI as common map services where they could simply enter a meaning address instead of latitude, longitude of destination. The response that is currently received contains coordinates, display name, and some other details of this address.

All four APIs were tested by Python HTTP session with JSON returned response. All test scripts and snippets of response are attached in the Appendix section.

Technology Stack

This section will define the technologies and tooling used to develop the program. It will also contain the requirements for the environment the program will be run in.

Programming languages

Primary programming language

The primary programming language for this project will be **C++**

Reasoning: It is the recommended programming language for the course. For object-oriented programming, C++ is one of the most suitable choices.

Secondary programming languages

The secondary programming languages used in this project are **SQL** and, if needed, **Python.**

Frameworks and libraries used

This section will discuss the individual libraries used in the project. Included will be the purpose, reasoning and technical information (e.g. versioning) of the libraries used.

UI

The UI of this project will be handled with **Qt with KDE Marble as the map rendering engine.**

Reasoning: We all are familiar with Qt from the programming courses, and it handles a lot of the heavy lifting of the general UI. KDE Marble is a plugin for Qt that handles map rendering and the associated extra operations (e.g. projections and stitching).

Version: Qt 6.12 (with standard support until 2031)

Networking

The networking of this project will be handled by **Boost.Beast.**

Reasoning: It is the C++ HTTP library, a gold standard for C++ networking, supports asynchronous operation and there is no reason in implementing our own networking library for a project that heavily relies on API calls. Our chosen APIs use HTTPS.

Local Database

The database operations for this project will be handled using **SQLite**.

Reasoning: We need a database for caching and avoiding rate-limits. SQLite is an industry standard and it comes as a drop-in header file. Additionally there is previous experience in the team with this database system.

Build System

Tooling: CMake

Reasoning: We could build the application via compiler-independent instruction. It could also automate unit testing, packaging and installations, making it more convenient for TA's grading.

The Usage of AI Tools in this Project

This section describes how AI tools are used throughout the project. AI tools are allowed in this project, but their use is limited and fully transparent. Every use of AI that affects the code or the documentation must be reported. A team member may only commit AI-generated code or text if they understand it and can explain it to other members or to the TA.

AI usage is documented in a separate wiki article in the project repository. Each entry in this log records the date, the team member who used the tool, the affected area of the project, a short description of what was done and how the output was utilized, the AI tool and model used, the prompt, and a link to the exact response of the model or to the saved conversation.

Other Tooling Used in the Project

Git is used for version control of the project. The project repository is hosted on GitHub, which serves as the central place for the team's code. In addition to storing the repository, GitHub is used for reviewing each other's changes before they are merged and for keeping the project wiki, which contains, for example, the AI usage log.

Figma is used for designing the user interface of the application and is used as a reference during the implementation of the UI.

Microsoft Teams is used for team collaboration and communication. The team's channels, meeting notes and project conventions are kept in Teams, and this design document is stored and edited together in the team's shared folder.

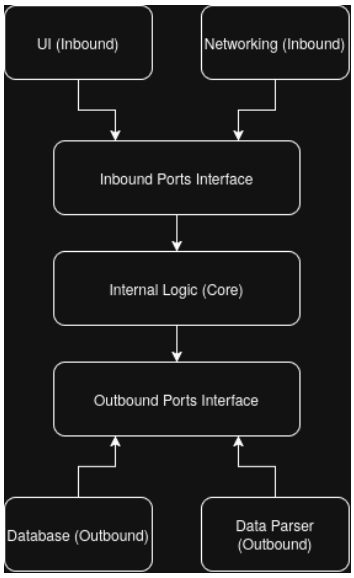
Environment Requirements

It should be implemented as an application that could run locally. To run this application, no account creation, API key creation or backend hosting is needed.

Architecture and Components

This section will first give an overview of the architecture of the program and then dive into the individual components and their purpose in the program.

Overview



The program will use a hexagonal architecture (Ports and Adapters architecture),

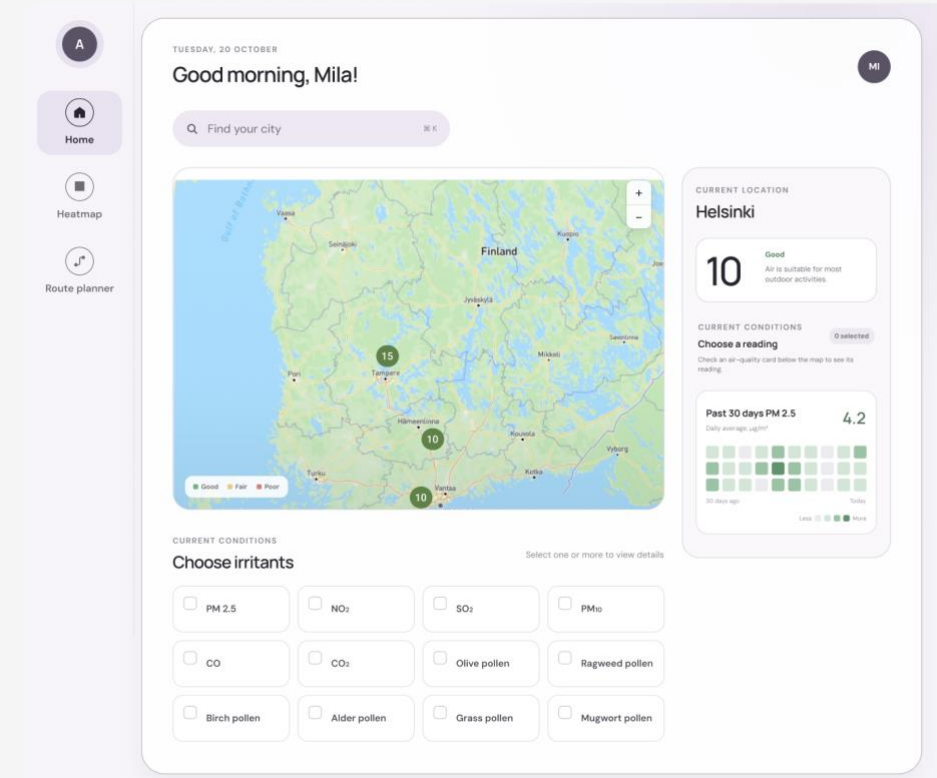
where the core logic of the application is completely independent of the components (outbound and inbound) that support it and, instead, serves two interfaces that the components work with. This keeps the system as a whole well decoupled and implements the dependency inversion principle.

UI Prototype

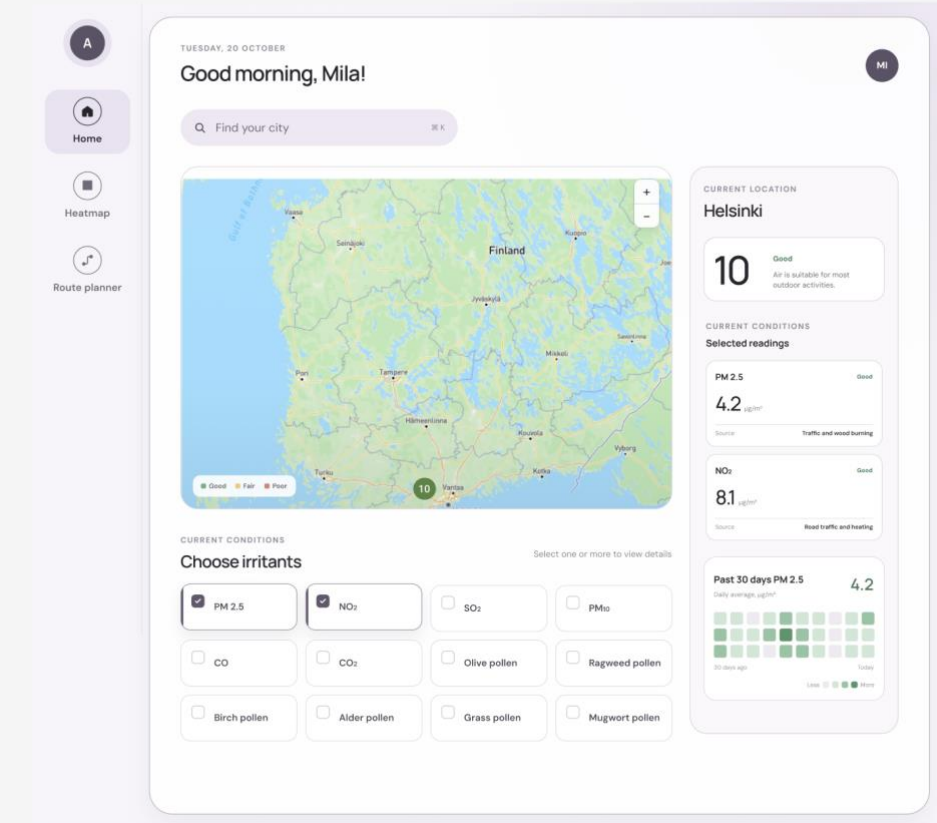
UI design:

https://www.figma.com/design/JLui3o7nn19J3cZYxL5xUf/software-design?node-id=0-1&t=tYV8WZpaQpKTybxP-1

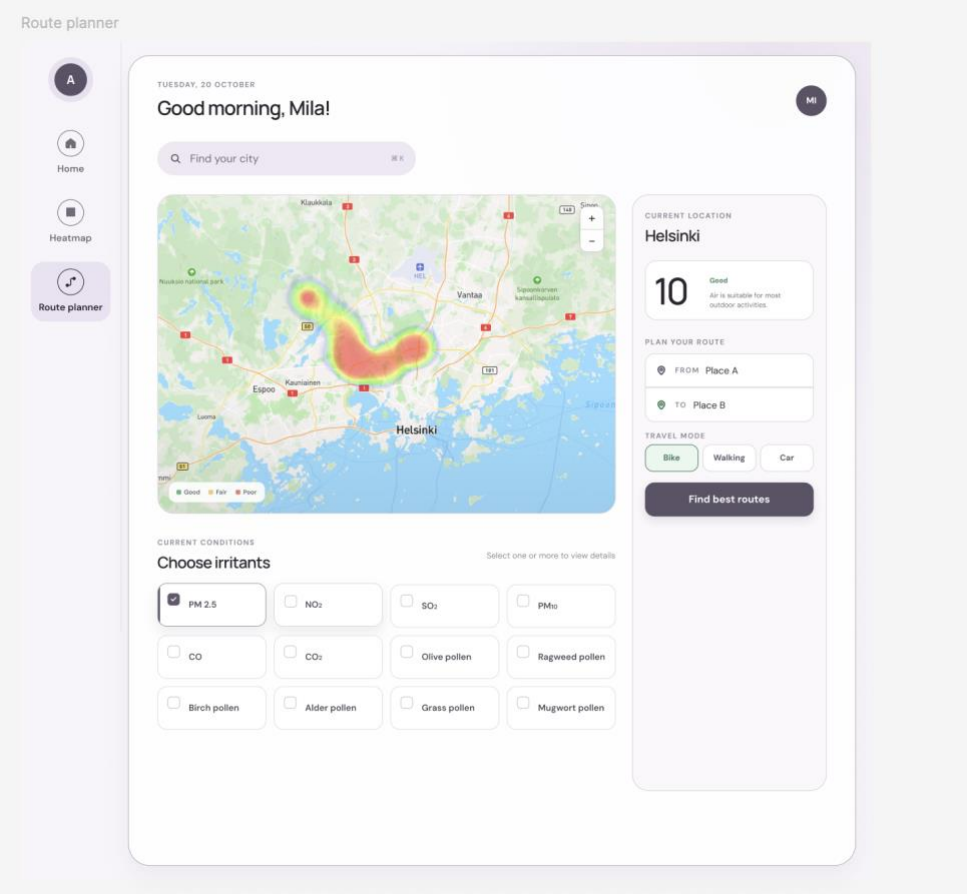
Home page - 1



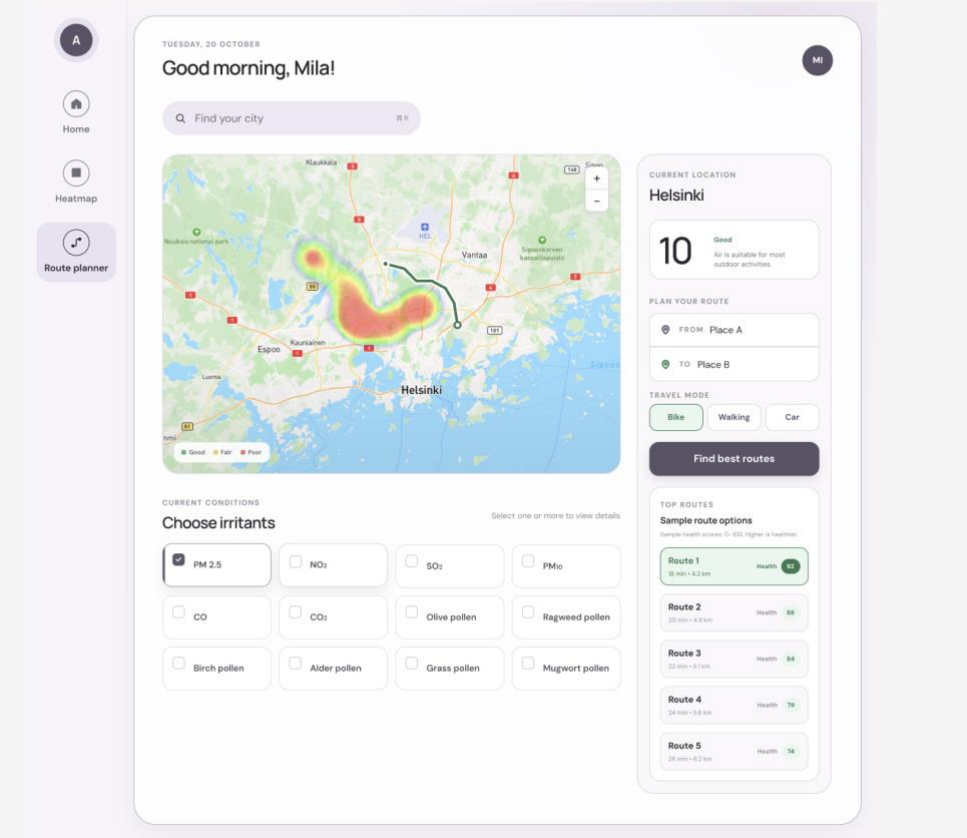
Home page - 2



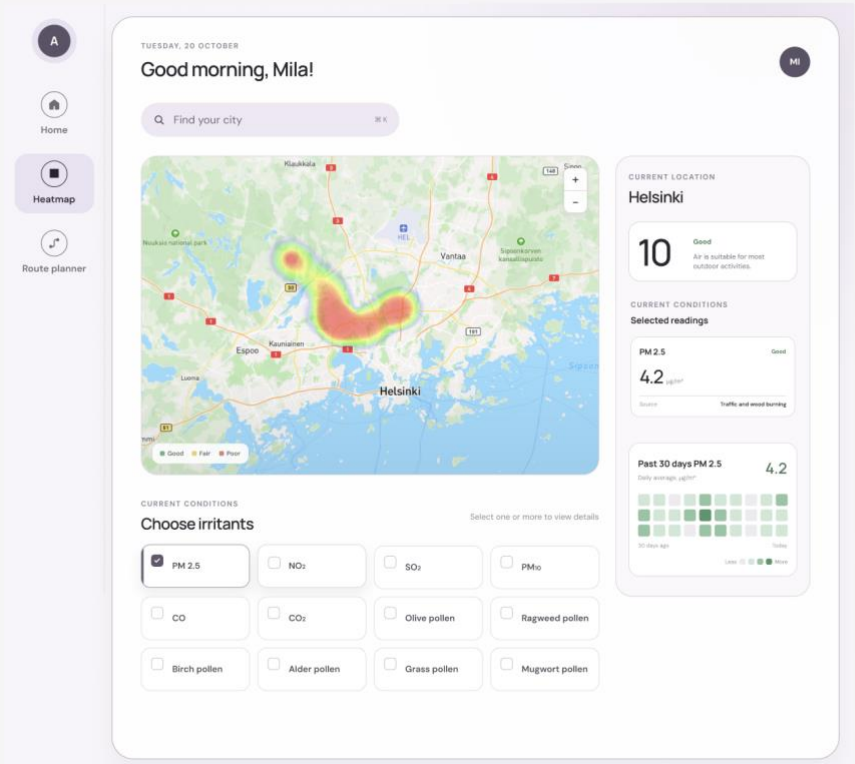
Route planner



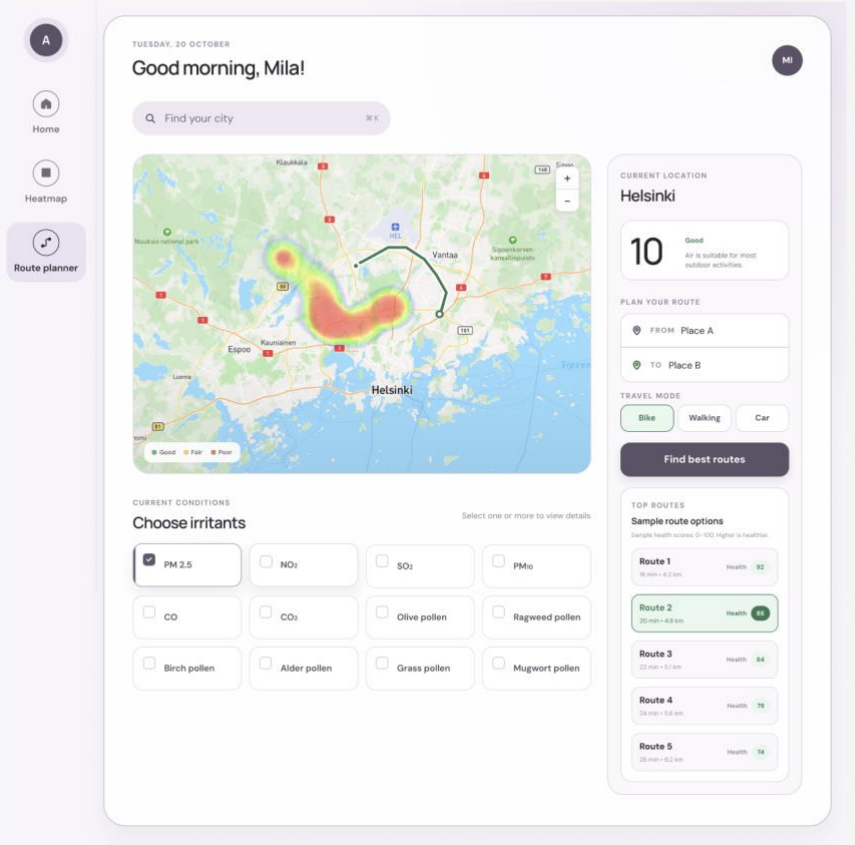
Route planner 2



Heatmap



Route planner 3



Member Involvement

This section contains a description and log of the work done by each member of the team.

Information for the TA: No details of the next phases of the project are known at this time, so it was agreed that the workload will be divided as we go. No project roles will be set, but each team member will be assigned a focus area in design/development. The areas and roles are intentionally kept flexible so that, if need be, other members can assist with work outside their own focus area. The following tables contain a high-level description of what every member does, and a more detailed internal log will be kept in the repository's DevOps documentation.

Everyone has participated in general ideating and drawing guidelines for the project together during our weekly meetings and between them.

Ville Ylönen

Primary Focus: TBA

Task	Date	Notes
Documentation setup	Weeks 40-42	Incremental creation and setup of documentation (meeting notes, project conventions and design document) and Teams group/channels.
Requirements gathering and initial technological decisions	Week 42	
Initial high-level architectural design	Week 43	

Mila Huovinen

Primary Focus: TBA

Task	Date	Notes
Group formation	Weeks 36-37	Moodle discussion, Telegram group generation, setting up the first meeting.
UI design	Week 43	Design created using Figma.

Van Bui

Primary Focus: TBA

Task	Date	Notes
UI design	Week 43	Design prototype using Figma
Refine and submit the final Design Document	Week 43	

Trang Vu

Primary Focus: TBA

Task	Date	Notes
Extending design documentation	18/10/2026	
APIs section and Appendix completion	20/10/2026	

Appendix

Appendix A: API Test Scripts and JSON responses

Open-Meteo Air Quality API

```
GET https://air-quality-api.open-meteo.com/v1/air-quality
```

```
import json
import requests_cache
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)

# Make sure all required weather variables are listed here
# The order of variables in hourly or daily is important to assign them correctly below
url = "https://air-quality-api.open-meteo.com/v1/air-quality"
params = {
    "latitude": 61.4981,    # Tampere railway station
    "longitude": 23.7737,   # Tampere railway station
    "hourly": ["pm10", "pm2_5", "carbon_monoxide", "carbon_dioxide", "nitrogen_dioxide",
"sulphur_dioxide", "ozone", "methane", "ammonia", "uv_index", "alder_pollen",
"birch_pollen", "grass_pollen", "olive_pollen", "mugwort_pollen", "ragweed_pollen",
"dust"],
    "start_date": "2026-10-18",
    "end_date": "2026-10-22",
}
response = retry_session.get(url, params = params)

with open("data/openmeteo_airquality.json", "w", encoding="utf-8") as file:
    json.dump(response.json(), file, indent=2, ensure_ascii=False)
```

JSON response


```
{
  "latitude": 61.5,
  "longitude": 23.8,
  "generationtime_ms": 0.8759498596191406,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 95.0,
  "hourly_units": {
    "time": "iso8601",
    "pm10": "µg/m³",
    "pm2_5": "µg/m³",
    "carbon_monoxide": "µg/m³",
    "carbon_dioxide": "ppm",
    "nitrogen_dioxide": "µg/m³",
    "sulphur_dioxide": "µg/m³",
    "ozone": "µg/m³",
    "methane": "µg/m³",
    "ammonia": "µg/m³",
    "uv_index": "",
    "alder_pollen": "grains/m³",
    "birch_pollen": "grains/m³",
    "grass_pollen": "grains/m³",
    "olive_pollen": "grains/m³",
    "mugwort_pollen": "grains/m³",
    "ragweed_pollen": "grains/m³",
```

```
    "dust": "µg/m³"
  },
  "hourly": {
    "time": [
      "2026-10-18T00:00",
      "2026-10-18T01:00",
      "2026-10-18T02:00",
      "2026-10-18T03:00",
      "2026-10-18T04:00",
```


Open-Meteo Weather Forecast API

```
GET https://api.open-meteo.com/v1/forecast
```

```
import json
import requests_cache
from retry_requests import retry

# Setup the Open-Meteo API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)

# Make sure all required weather variables are listed here
# The order of variables in hourly or daily is important to assign them correctly below
url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 61.4981,    # Tampere railway station
    "longitude": 23.7737,   # Tampere railway station
    "daily": "uv_index_max",
    "hourly": ["temperature_2m", "apparent_temperature"],
    "current": ["temperature_2m", "apparent_temperature", "relative_humidity_2m"],
    "timezone": "GMT",
}
response = retry_session.get(url, params = params)

with open("data/openmeteo_weather.json", "w", encoding="utf-8") as file:
    json.dump(response.json(), file, indent=2, ensure_ascii=False)
```

JSON response


```
{
  "latitude": 61.49773,
  "longitude": 23.779099,
  "generationtime_ms": 0.202178955078125,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 95.0,
  "current_units": {
    "time": "iso8601",
    "interval": "seconds",
    "temperature_2m": "°C",
    "apparent_temperature": "°C",
    "relative_humidity_2m": "%"
  },
  "current": {
    "time": "2026-10-20T16:15",
    "interval": 900,
    "temperature_2m": 4.7,
    "apparent_temperature": 1.8,
    "relative_humidity_2m": 86
  },
```

```
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C",
    "apparent_temperature": "°C"
  },
  "hourly": {
    "time": [
      "2026-10-20T00:00",
      "2026-10-20T01:00",
      "2026-10-20T02:00",
      "2026-10-20T03:00",
      "2026-10-20T04:00",
      "2026-10-20T05:00",
```


Valhalla API

```
POST https://valhalla1.openstreetmap.de/route
```

```
import json
import requests_cache
from retry_requests import retry

# Setup the API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)

url = "https://valhalla1.openstreetmap.de/route"
locations = [
    {"lat": 61.4981, "lon": 23.7737},  # Tampere railway station
    {"lat": 61.4500, "lon": 23.8500},  # Hervanta
]

costings = ["auto", "bicycle", "pedestrian"]

for costing in costings:
    body = {"locations": locations, "costing": costing, "alternates": 3}
    response = retry_session.post(url, json = body)

    with open(f"data/valhalla_{costing}.json", "w", encoding = "utf-8") as file:
        json.dump(response.json(), file, indent = 2, ensure_ascii = False)
```

JSON response


```
{
  "trip": {
    "locations": [
      {
        "type": "break",
        "lat": 61.4981,
        "lon": 23.7737,
        "side_of_street": "left",
        "original_index": 0
      },
      {
        "type": "break",
        "lat": 61.45,
        "lon": 23.85,
        "original_index": 1
      }
    ],
    "legs": [
      {
        "maneuvers": [
          {
            "type": 3,
```

```
            "instruction": "Drive north on Ratapihankatu.",
            "verbal_succinct_transition_instruction": "Drive north. Then Enter the roundabout and take
the 2nd exit onto Ratapihankatu.",
            "verbal_pre_transition_instruction": "Drive north on Ratapihankatu. Then Enter the
roundabout and take the 2nd exit onto Ratapihankatu.",
            "verbal_post_transition_instruction": "Continue for 200 meters.",
            "street_names": [
              "Ratapihankatu"
            ],
            "bearing_after": 359,
            "time": 37.355,
            "length": 0.228,
            "cost": 43.713,
            "begin_shape_index": 0,
            "end_shape_index": 16,
            "verbal_multi_cue": true,
            "travel_mode": "drive",
            "travel_type": "car"
          },
```


Nominatim API

```
GET https://nominatim.openstreetmap.org/search
```

```
import json
import requests_cache
from retry_requests import retry

# Setup the API client with cache and retry on error
cache_session = requests_cache.CachedSession('.cache', expire_after = 3600)
retry_session = retry(cache_session, retries = 5, backoff_factor = 0.2)

url = "https://nominatim.openstreetmap.org/search"

params = {
    "q": "Hämeenkatu 1, Tampere",
    "countrycodes": "fi",
    "format": "jsonv2",
    "addressdetails": 1,
    "limit": 1,
}

headers = {"User-Agent": "CommutePlanner/1.0 (name@tuni.fi)"}

response = retry_session.get(url, params=params, headers=headers)
with open("data/nominatim_geocoding.json", "w", encoding="utf-8") as file:
    json.dump(response.json(), file, indent=2, ensure_ascii=False)
```

JSON response


```
[
  {
    "place_id": 166083298,
    "licence": "Data © OpenStreetMap contributors, ODbL 1.0. http://osm.org/copyright",
    "osm_type": "node",
    "osm_id": 139812404,
    "lat": "61.4979285",
    "lon": "23.7716105",
    "category": "tourism",
    "type": "hotel",
    "place_rank": 30,
    "importance": 7.226901655599325e-05,
    "addresstype": "tourism",
    "name": "Scandic Tampere City",
    "display_name": "Scandic Tampere City, 1, Hämeenkatu, Kyttälä, Kyttälä A, Keskustan suuralue,
Tampere, Tampereen seutukunta, Pirkanmaa, Manner-Suomi, 33100, Suomi / Finland",
```

```
    "address": {
      "tourism": "Scandic Tampere City",
      "house_number": "1",
      "road": "Hämeenkatu",
      "suburb": "Kyttälä",
      "city_district": "Keskustan suuralue",
      "city": "Tampere",
      "municipality": "Tampereen seutukunta",
      "state": "Pirkanmaa",
      "ISO3166-2-lvl4": "FI-11",
      "region": "Manner-Suomi",
      "postcode": "33100",
      "country": "Suomi / Finland",
      "country_code": "fi"
    },
    "boundingbox": [
      "61.4978785",
      "61.4979785",
      "23.7715605",
      "23.7716605"
    ]
  }
]
```